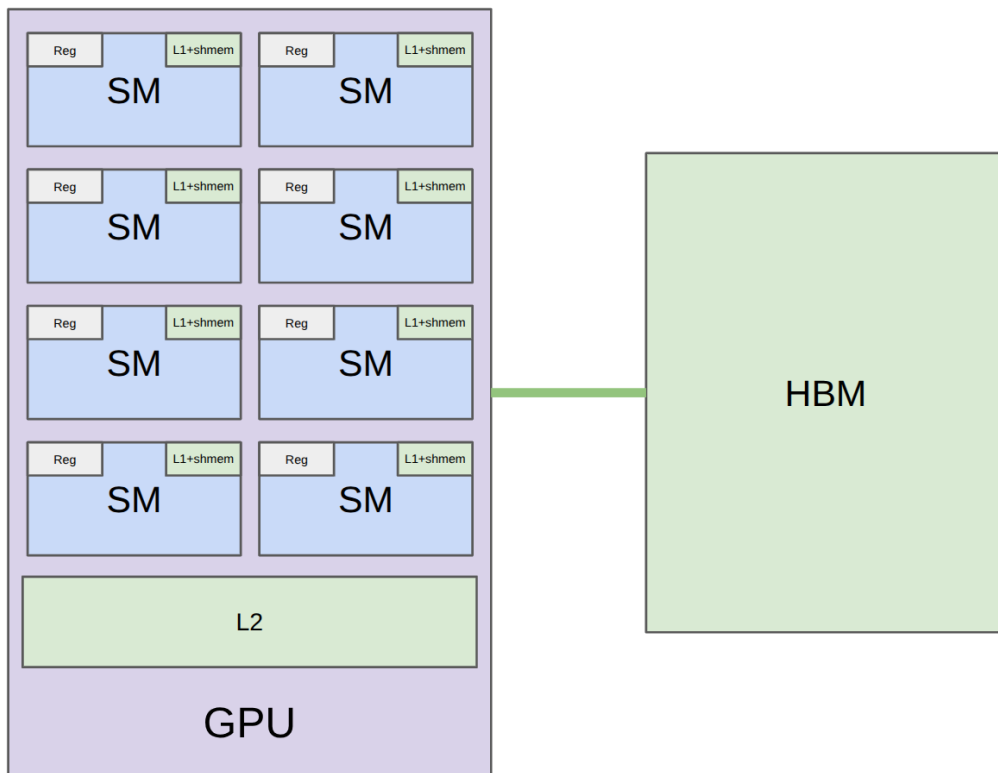


CS336 2026 Lecture 6: Benchmarking, Profiling 与 Triton Kernels

基于 Stanford CS336 Spring 2026 官方可执行讲义整理

2026 年春季



课程/频道: Stanford CS336

发布日期: Spring 2026

材料类型: 官方可执行讲义 lecture06-slides.py

课程主页: <https://cs336.stanford.edu/>

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 本讲主线：正确性靠编程模型，性能靠硬件模型	3
2 GPU recap：从硬件到编程模型	3
2.1 本章小结	6
3 Benchmarking 与 Profiling：先量再改	6
3.1 本章小结	6
4 GeLU case study：naive、builtin、compiled	7
4.1 本章小结	7
5 Triton mental model	7
5.1 本章小结	8
6 Triton softmax：row fits in one block	8
6.1 本章小结	9
7 Triton row sum：row larger than block	9
7.1 本章小结	10
8 Triton matmul + ReLU：tiling and fusion	10
8.1 本章小结	11
9 补充推导：从 profiling 到 Triton kernel 的完整 workflow	11
9.1 GPU hardware ledger	11
9.2 Benchmarking 的最小正确性条件	11
9.3 Profiling 如何读 kernel 名	12
9.4 GeLU fusion 的 HBM 账本	12
9.5 Triton GeLU kernel 逐行解释	13
9.6 Softmax 为什么适合 row-wise block	13
9.7 Row sum：当一行放不进一个 block	13
9.8 Tiled matmul + ReLU 的完整账本	14
9.9 本章小结	14
10 源码节点全覆盖索引	14
11 更细的硬件解释：为什么这些细节会影响性能	15
11.1 Warps、divergence 与 occupancy	15
11.2 Bank conflicts 与 swizzling	15
11.3 Memory coalescing 与 HBM transaction	15
12 Triton kernel 参数表	15
13 扩展讲解：从源码到性能判断	16
13.1 为什么 GeLU 是第一个好例子	16
13.2 Profiler 表应该怎么看	16

13.3 Triton masks 的必要性	17
13.4 Softmax 的读写账本	17
13.5 Matmul tiling 的指针结构	17
13.6 本章小结	18
14 Assignment 2 视角: 从 correctness 到 performance	18
14.1 从本讲走向下一讲	18
15 总结与延伸	19
15.1 拓展阅读	19

1 本讲主线：正确性靠编程模型，性能靠硬件模型

Lecture 6 接在 GPU 课之后，目标是把“知道 GPU 很快”推进到“知道怎么测、怎么定位、怎么写 kernel”。核心循环是：benchmark 看到端到端时间，profile 看到时间花在哪里，然后用更贴近硬件的数据组织和 kernel 写法修正瓶颈。

本讲的闭环

编程模型给正确性，硬件模型给性能，benchmark/profiling 给证据，Triton 给可控的 block-level 实现。没有测量就改 kernel，基本是在猜。

术语消化：GPU kernel 基础词汇

- **Kernel**: 在 GPU 上执行的函数。
- **HBM**: High Bandwidth Memory, GPU 的大容量全局显存，带宽高但仍远慢于片上 register/shared memory。
- **Thread block / CTA**: 一组线程，被调度到同一个 SM，共享 shared memory。
- **Warp**: 通常 32 个 lockstep 执行的 threads；分支不同会产生 control divergence。
- **Occupancy**: SM 上活跃 warps/blocks 的比例，受 registers、shared memory、block size 限制。
- **Bank conflict**: 多个 threads 同时访问 shared memory 同一个 bank，访问被串行化。
- **Memory coalescing**: warp 中线程访问连续 HBM cache line，合并成高效 memory transaction。
- **Tiling**: 把大矩阵分块搬到 shared memory 中复用。
- **Fusion**: 把多个操作合并进一个 kernel，减少 HBM 往返。

符号说明：kernel 性能公式

本讲出现的 N 表示向量或矩阵维度， M, N, K 在 matmul 中分别表示 $A \in \mathbb{R}^{M \times K}$ 、 $B \in \mathbb{R}^{K \times N}$ 、 $C \in \mathbb{R}^{M \times N}$ 的维度；FLOPs 是浮点操作数，bytes moved 是 HBM 读写量。Arithmetic intensity $I = \text{FLOPs}/\text{bytes}$ 越高，越容易让 compute units 忙起来。

2 GPU recap：从硬件到编程模型

Source coverage: review_of_gpus. 本节覆盖的核心概念包括：SM/warp/block/shared memory/bank conflict/coalescing/occupancy。

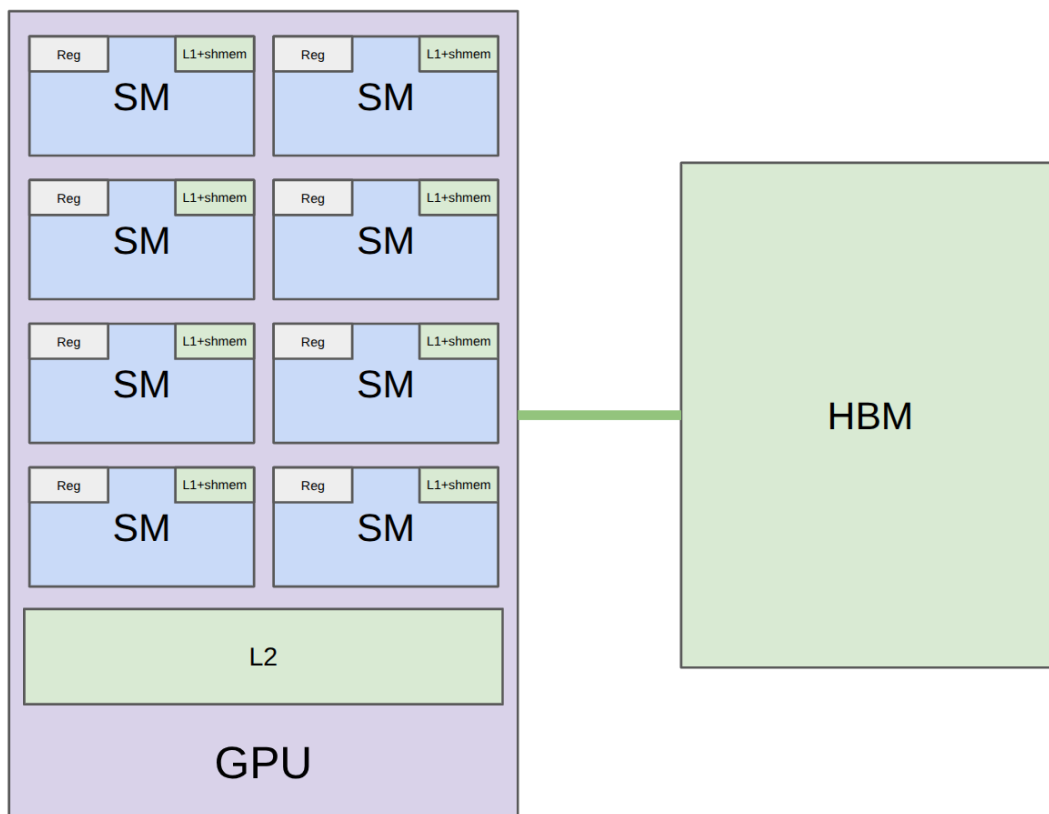


图 1: gpu hardware.png

来源: CS336 Spring 2026 Lecture 6 官方可执行讲义。

读图: gpu hardware.png

这张图用于把抽象的 kernel 代码连接到硬件执行。读图时先看数据位于 HBM、shared memory 还是 registers, 再看线程/blocks 如何映射到 SM。性能优化的关键是让数据在更靠近计算单元的位置被复用, 减少慢速 HBM 往返。

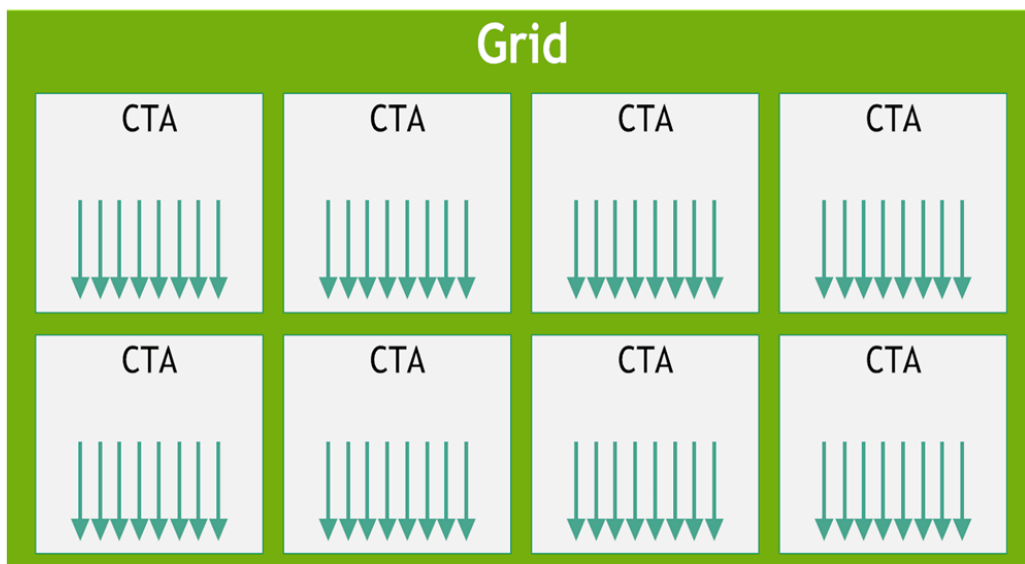


图 2: cuda grid.png

来源: CS336 Spring 2026 Lecture 6 官方可执行讲义。

读图: cuda grid.png

这张图用于把抽象的 kernel 代码连接到硬件执行。读图时先看数据位于 HBM、shared memory 还是 registers, 再看线程/blocks 如何映射到 SM。性能优化的关键是让数据在更靠近计算单元的位置被复用, 减少慢速 HBM 往返。

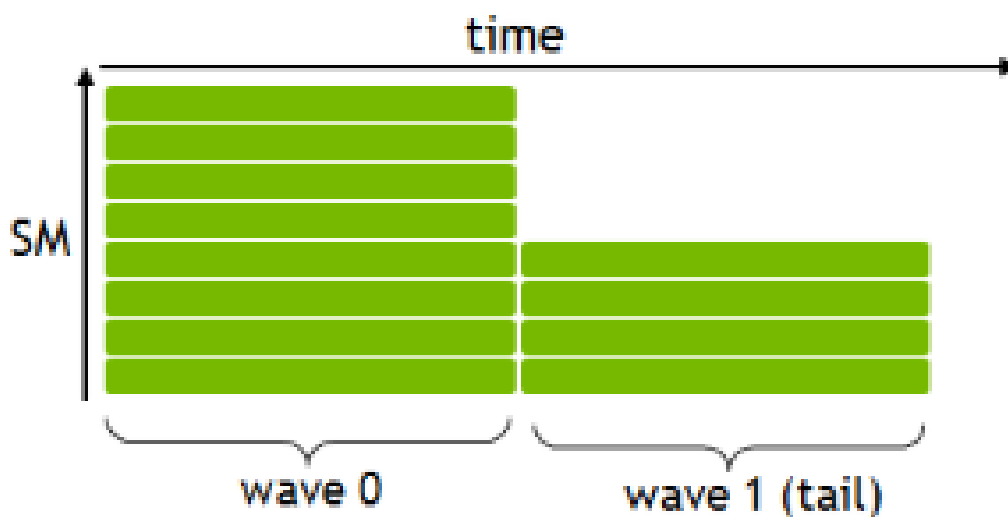


图 3: block occupancy.png

来源: CS336 Spring 2026 Lecture 6 官方可执行讲义。

读图: block occupancy.png

这张图用于把抽象的 kernel 代码连接到硬件执行。读图时先看数据位于 HBM、shared memory 还是 registers, 再看线程/blocks 如何映射到 SM。性能优化的关键是让数据在更靠近计算单元的位置被复用, 减少慢速 HBM 往返。

2.1 本章小结

本节说明了一个 kernel optimization 的共同模式: 先确认语义正确, 再测时间, 再看 profiler, 最后用 block、shared memory、tiling、fusion 或 layout 调整降低数据移动。

3 Benchmarking 与 Profiling: 先量再改

Source coverage: benchmarking/profiling. 本节覆盖的核心概念包括: benchmarking, profiling, CUDA synchronize, CUDA events, torch profiler。

方法	回答的问题	常见陷阱
Benchmark	这个操作端到端多快? 随 shape 如何变化?	未同步 CUDA、warmup 不足、只测一次。
Profiler	时间花在哪些 kernels 上? 实际调用了什么?	只看总时间, 不看 kernel 名和 shape。
Nsight	更底层地看 occupancy、memory、warp stalls	信息量大, 需要明确假设再查。

Listing 1: CUDA event benchmark 的关键结构

```
1 for _ in range(num_warmups):
2     run()
3 torch.cuda.synchronize()
4 start_event.record()
5 run()
6 end_event.record()
7 torch.cuda.synchronize()
8 time_ms = start_event.elapsed_time(end_event)
```

benchmark 必须同步

CUDA kernel launch 是异步的。如果不调用 `torch.cuda.synchronize()` 或 CUDA events, 测到的可能只是 CPU 发起 kernel 的时间, 而不是 GPU 真正执行时间。

3.1 本章小结

本节说明了一个 kernel optimization 的共同模式: 先确认语义正确, 再测时间, 再看 profiler, 最后用 block、shared memory、tiling、fusion 或 layout 调整降低数据移动。

4 GeLU case study: naive、builtin、compiled

Source coverage: `naive_vs_builtin_vs_compiled_gelu`. 本节覆盖的核心概念包括: kernel fusion, `torch.compile`, HBM reads/writes。

Listing 2: naive GeLU 的问题: 多个小操作导致多次 HBM 往返

```
1 def naive_gelu(x):  
2     return 0.5 * x * (1 + torch.tanh(0.79788456 * (x + 0.044715 * x * x * x)))
```

为什么 builtin/compiled GeLU 更快

Naive PyTorch 写法会拆成多个 primitive kernels, 中间结果反复写回 HBM。Builtin GeLU 或 `torch.compile` 后的 Triton kernel 能把多个 pointwise 操作融合成一次读、一次写, 减少 HBM traffic。

4.1 本章小结

本节说明了一个 kernel optimization 的共同模式: 先确认语义正确, 再测时间, 再看 profiler, 最后用 block、shared memory、tiling、fusion 或 layout 调整降低数据移动。

5 Triton mental model

Source coverage: `triton_introduction` / `triton_gelu_example`. 本节覆盖的核心概念包括: Triton program, block, mask, pointer offsets, PTX。

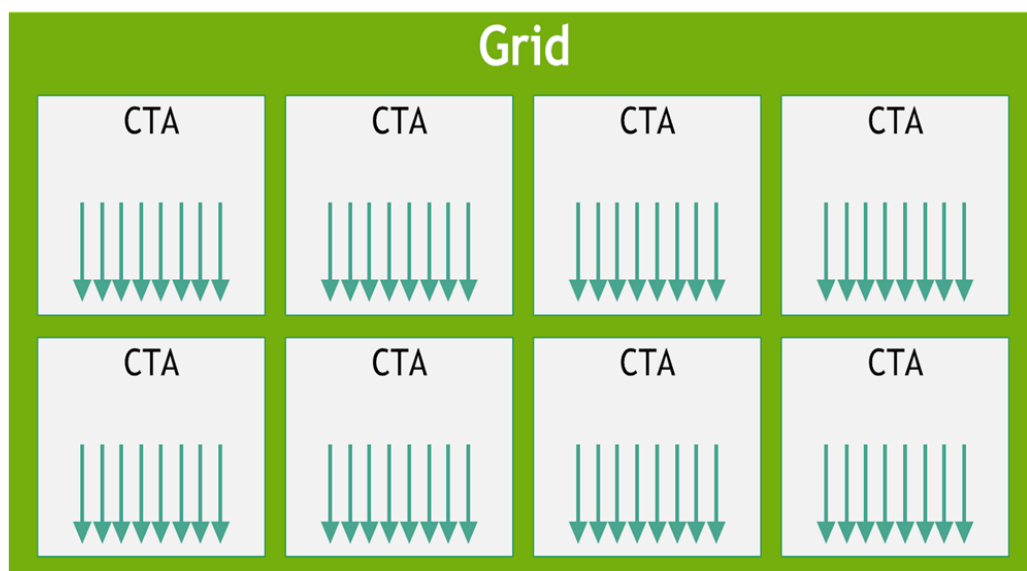


图 4: cuda.grid.png

来源: *CS336 Spring 2026 Lecture 6* 官方可执行讲义。

读图: cuda_grid.png

这张图用于把抽象的 kernel 代码连接到硬件执行。读图时先看数据位于 HBM、shared memory 还是 registers, 再看线程/blocks 如何映射到 SM。性能优化的关键是让数据在更靠近计算单元的位置被复用, 减少慢速 HBM 往返。

Listing 3: Triton block-level 思维

```
1 pid = tl.program_id(axis=0)
2 offsets = pid * BLOCK_SIZE + tl.arange(0, BLOCK_SIZE)
3 mask = offsets < num_elements
4 x = tl.load(x_ptr + offsets, mask=mask)
5 tl.store(y_ptr + offsets, y, mask=mask)
```

Triton 与 CUDA 的差异

CUDA 通常让你描述每个 thread 做什么; Triton 更像描述一个 program/block 如何处理一块数据。它牺牲部分底层控制, 换来更高层、更适合深度学习 kernel 的表达。

5.1 本章小结

本节说明了一个 kernel optimization 的共同模式: 先确认语义正确, 再测时间, 再看 profiler, 最后用 block、shared memory、tiling、fusion 或 layout 调整降低数据移动。

6 Triton softmax: row fits in one block

Source coverage: `triton_softmax_example`. 本节覆盖的核心概念包括: row-wise reduction, max subtraction, exp, normalization.

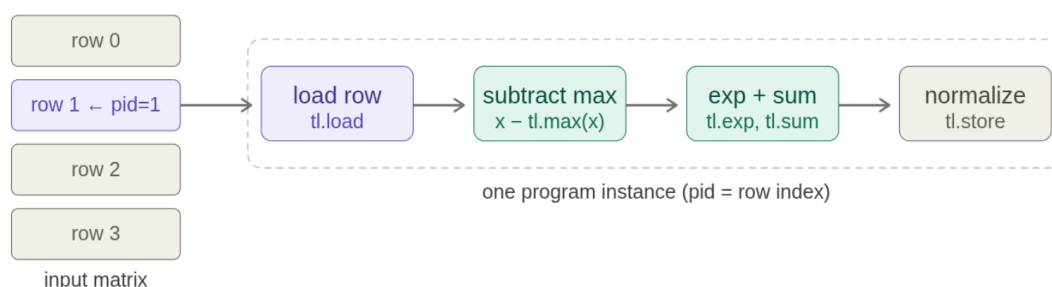


图 5: triton_softmax.png

来源: CS336 Spring 2026 Lecture 6 官方可执行讲义。

读图: triton_softmax.png

这张图用于把抽象的 kernel 代码连接到硬件执行。读图时先看数据位于 HBM、shared memory 还是 registers, 再看线程/blocks 如何映射到 SM。性能优化的关键是让数据在更靠近计算单元的位置被复用, 减少慢速 HBM 往返。

$$\text{softmax}(x_i) = \frac{e^{x_i - m}}{\sum_j e^{x_j - m}}, \quad m = \max_j x_j.$$

fused softmax 的资源意义

Naive softmax 需要 max、subtract、exp、sum、divide 多个 pass。若一整行能放进一个 Triton block，就可以在 shared memory/register 中完成 reduction，只读写 HBM 各一次左右。

6.1 本章小结

本节说明了一个 kernel optimization 的共同模式：先确认语义正确，再测时间，再看 profiler，最后用 block、shared memory、tiling、fusion 或 layout 调整降低数据移动。

7 Triton row sum: row larger than block

Source coverage: `triton_row_sum_example`. 本节覆盖的核心概念包括：tiling over columns, accumulators, final reduction。

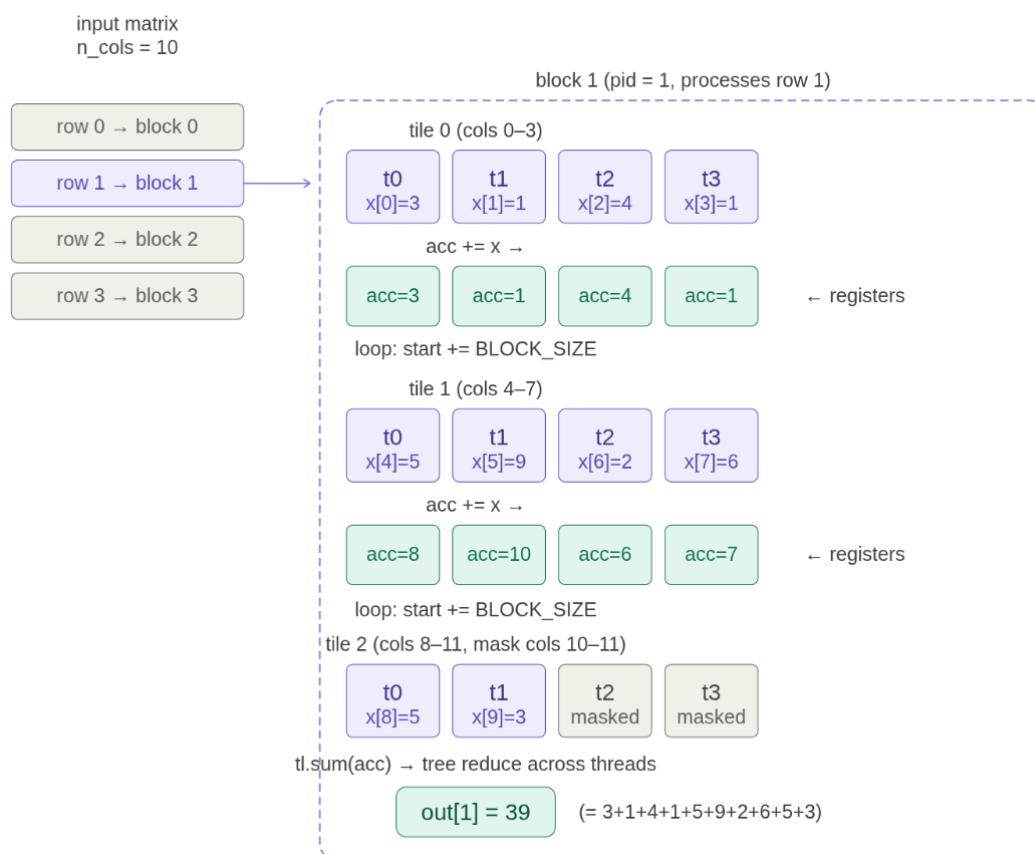


图 6: triton row sum.png

来源: CS336 Spring 2026 Lecture 6 官方可执行讲义。

读图: triton row sum.png

这张图用于把抽象的 kernel 代码连接到硬件执行。读图时先看数据位于 HBM、shared memory 还是 registers，再看线程/blocks 如何映射到 SM。性能优化的关键是让数据在更靠近计算单元的位置被复用，减少慢速 HBM 往返。

row 不适合一个 block 时怎么办

如果一行太长，不能一次放进一个 block，就把行切成 tiles。每个 program 遍历多个 tiles，局部累加到 accumulator，最后再做 block 内 reduction。这是 tiling 思想的最小例子。

7.1 本章小结

本节说明了一个 kernel optimization 的共同模式：先确认语义正确，再测时间，再看 profiler，最后用 block、shared memory、tiling、fusion 或 layout 调整降低数据移动。

8 Triton matmul + ReLU: tiling and fusion

本节覆盖源码函数 `triton_matmul_relu_example`。核心概念包括 tiled matmul、`tl.dot`、shared memory reuse 和 fused activation。

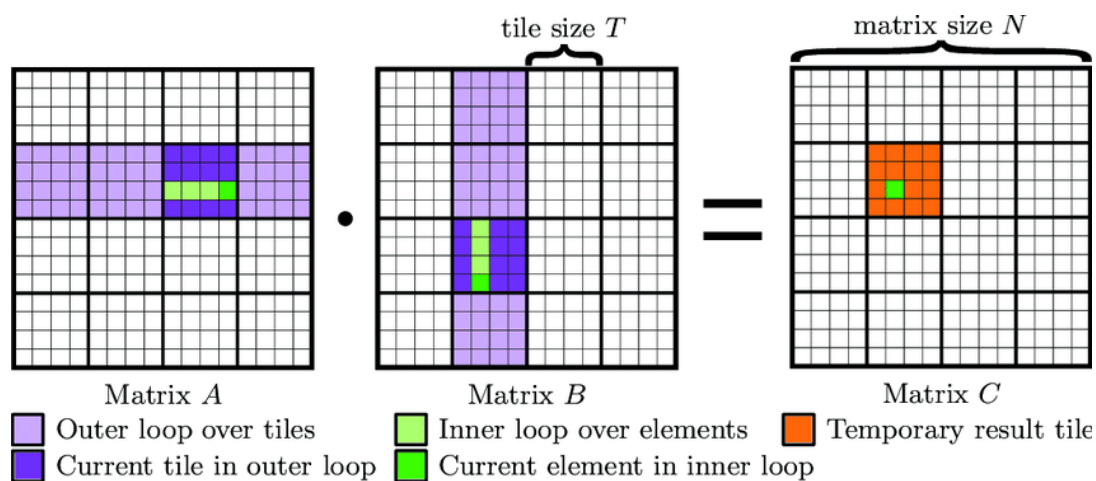


图 7: gemm tiled.png

来源: CS336 Spring 2026 Lecture 6 官方可执行讲义。

读图: gemm tiled.png

这张图用于把抽象的 kernel 代码连接到硬件执行。读图时先看数据位于 HBM、shared memory 还是 registers，再看线程/blocks 如何映射到 SM。性能优化的关键是让数据在更靠近计算单元的位置被复用，减少慢速 HBM 往返。

Listing 4: Tiled matmul kernel 的核心循环

```

1 for k in range(0, K, BLOCK_K):
2     a = tl.load(a_ptrs, mask=...)
3     b = tl.load(b_ptrs, mask=...)
4     acc += tl.dot(a, b)
5     a_ptrs += BLOCK_K * stride_ak
6     b_ptrs += BLOCK_K * stride_bk
7 acc = tl.maximum(acc, 0.0) # fused ReLU

```

Matmul + ReLU fusion

如果下一步马上做 ReLU, 就没有必要先把 matmul 输出写回 HBM, 再读出来做 ReLU。把 ReLU 放在 matmul kernel 尾部, 就是典型 fusion。

8.1 本章小结

本节说明了一个 kernel optimization 的共同模式: 先确认语义正确, 再测时间, 再看 profiler, 最后用 block、shared memory、tiling、fusion 或 layout 调整降低数据移动。

9 补充推导: 从 profiling 到 Triton kernel 的完整 workflow

9.1 GPU hardware ledger

硬件层级	典型容量/特征	优化含义
Registers	每个 thread 私有, 极快	register 用太多会降低 occupancy。
Shared memory / L1	每个 SM 内, 快但小	适合 tile、row reduction、softmax 等 block 内复用。
L2 cache	全 GPU 共享缓存	对跨 block 的重复访问有帮助, 但不可完全依赖。
HBM	GPU global memory, 大容量高带宽	仍远慢于片上存储; kernel 优化要减少 HBM 往返。
Tensor cores	专用矩阵硬件	需要 shape/dtype/layout 匹配, 才能达到高吞吐。

occupancy 不是越高越好

高 occupancy 能隐藏 HBM latency, 但如果每个 thread 做更多 work、复用更多数据, 较低 occupancy 也可能更快。正确问题不是“occupancy 是否最大”, 而是瓶颈是不是因为可运行 warps 不够。

9.2 Benchmarking 的最小正确性条件

Listing 5: Benchmarking checklist

```

1 # 1. Warm up first to avoid compilation / cache effects.
2 for _ in range(num_warmups):
3     run()
4 torch.cuda.synchronize()
5
6 # 2. Use CUDA events for GPU timing.
7 start = torch.cuda.Event(enable_timing=True)
8 end = torch.cuda.Event(enable_timing=True)
9 start.record()
10 run()
11 end.record()
12 torch.cuda.synchronize()
13 time_ms = start.elapsed_time(end)

```

为什么第一次运行不能计入 benchmark

第一次运行可能包含 CUDA context 初始化、kernel compilation、allocator warmup、cache miss 等开销。我们关心 steady-state kernel performance，所以需要 warmup，并且要用多次 trial 看方差。

9.3 Profiling 如何读 kernel 名

Profiler 不只是给总时间，还暴露实际调用的 CUDA kernels。例如一个名字里可能出现：

- cutlass: NVIDIA 线性代数 kernel library。
- sm100: 对应 Blackwell/B200 架构。
- f32: 输入/输出/accumulator dtype 信息。
- 64x64x16: tile shape, 暗示 block 内计算组织。

profile 的核心价值

Benchmark 告诉你慢，profile 告诉你为什么慢。比如同样是 matmul，小 shape 可能调用 SIMT kernel，大 shape 才调用 tensor-core-friendly GEMM；同样是 GeLU，naive 写法可能拆成多个 pointwise kernels，compiled 版本可能融合成 Triton kernel。

9.4 GeLU fusion 的 HBM 账本

Naive GeLU 公式是：

$$\text{GeLU}(x) \approx 0.5x \left(1 + \tanh \left(0.79788456(x + 0.044715x^3) \right) \right).$$

若每个子表达式都变成单独 kernel，可能产生多次 HBM read/write。Fused kernel 则把中间值保留在 registers 中：读一次 x ，计算多个子表达式，写一次 y 。

实现	kernel 形态	资源效果
----	-----------	------

Naive PyTorch	多个 pointwise kernels	多次 HBM 往返, launch overhead 多。
Builtin GeLU	库内 fused kernel	减少中间读写。
<code>torch.compile</code>	编译生成 fused Triton kernel	保留 Python 语义, 获得接近手写 kernel 的数据移动模式。

9.5 Triton GeLU kernel 逐行解释

Listing 6: Triton GeLU kernel 核心结构

```

1 pid = tl.program_id(axis=0)
2 start = pid * BLOCK_SIZE
3 offsets = start + tl.arange(0, BLOCK_SIZE)
4 mask = offsets < num_elements
5 x = tl.load(x_ptr + offsets, mask=mask)
6 # compute gelu in registers
7 ...
8 tl.store(y_ptr + offsets, y, mask=mask)

```

Triton program 的心智模型

每个 Triton program 类似一个 thread block, 负责一段 offsets。Mask 处理尾部不满 block 的元素。‘tl.load’ 从 global memory 读入向量, 后续计算在 registers 中完成, ‘tl.store’ 写回。这个模型比 CUDA thread-level 更粗, 但足以表达很多 ML kernels。

9.6 Softmax 为什么适合 row-wise block

Softmax 对每一行独立: 先求最大值 m , 再计算指数和归一化。

$$\text{softmax}(x_i) = \frac{e^{x_i - m}}{\sum_j e^{x_j - m}}, \quad m = \max_j x_j.$$

若一行能放进一个 block, Triton 可以一次读入整行, 在 block 内完成 max、exp、sum、divide, 再写回输出。这样避免 naive 版本多次对同一行读写 HBM。

softmax 的数值稳定性

先减去行最大值 m 不是性能技巧, 而是数值稳定技巧。否则 e^{x_i} 可能 overflow。性能优化不能破坏这种稳定性约束。

9.7 Row sum: 当一行放不进一个 block

Row sum 是 softmax 的简化版, 用来讲解“行太长”的情形。如果 $N = 4096$ 而 block size 是 1024, 一行需要 4 个 tiles。每个 program 在 tile loop 中累加 partial sum, 最后做 block 内 reduction。

Row sum 是 baby tiling

它没有 matmul 那么复杂, 但已经包含 tiling 的核心: 把大数据拆成可放进 block 的块, 循环加载, 局部累加, 最后归约。这是理解 FlashAttention 和 tiled matmul 的前置台阶。

9.8 Tiled matmul + ReLU 的完整账本

Naive matmul 对每个输出元素 C_{mn} 遍历 k ，不断从 HBM 读 A_{mk} 和 B_{kn} 。相邻输出元素会重复用到同一行 A 或同一列 B。如果直接从 HBM 重复读，arithmetic intensity 低。

Tiling 做法是：

1. 选择一个 C 的 output tile。
2. 依次加载对应的 A tile 和 B tile 到 shared memory。
3. 用 `tl.dot` 在 tile 上累积 partial sums。
4. 在写回 C 之前顺手做 ReLU，完成 fusion。

为什么 tile size 决定性能

Tile 太小，数据复用不够；tile 太大，shared memory/register 不够、occupancy 下降。高性能 GEMM kernel 的核心就是在 tile shape、register pressure、memory coalescing、tensor core layout 之间找平衡。

9.9 本章小结

这一节把源码中的 kernel examples 连接成完整工作流：GeLU 展示 pointwise fusion，softmax 展示 row-wise reduction，row sum 展示跨 tile accumulation，matmul+ReLU 展示 shared-memory tiling 和 activation fusion。四者共同说明：Triton 的价值在于让你以 block 为单位显式组织数据移动。

10 源码节点全覆盖索引

Lecture 6 是 executable source，真正的“slide-complete”不是逐页 PDF，而是逐个教学函数覆盖。下面这张表说明每个函数在讲义中的位置和处理方式。

源码函数	教学目标	讲义处理
<code>review_of_gpus</code>	复习硬件与编程模型	GPU 图、grid 图、block occupancy 图、术语表。
<code>benchmarking</code>	端到端计时	CUDA events、warmup、synchronize 代码。
<code>profiling</code>	看 kernel 级时间分布	profiler 表解释、kernel name 解读。
GeLU variants	fusion 的性能收益	GeLU 公式、naive/builtin/compiled 对比。
Triton GeLU	elementwise kernel	block offsets、mask、load/store、PTX 观察。
Triton softmax	row-wise reduction	max-subtract-exp-sum-divide 的 fused row kernel。
Triton row sum	row too large for one block	tile loop、accumulator、final reduction。
Triton matmul ReLU	tiled GEMM + fusion	tile pointers、 <code>tl.dot</code> 、fused ReLU。

这张索引表的目的

它是本讲的 coverage matrix 的正文版本。读者可以看到：本讲不是泛泛讲 Triton，而是沿源码中的每个教学函数建立 benchmark → profile → optimize → write kernels 的完整链条。

11 更细的硬件解释：为什么这些细节会影响性能

11.1 Warps、divergence 与 occupancy

Warp 是 GPU 执行的基本调度粒度。一个 warp 内的 32 个 threads 通常执行同一条指令。如果一半 threads 走 if 分支 A，另一半走分支 B，硬件往往需要串行执行 A 和 B，这叫 control divergence。

Occupancy 则描述一个 SM 上有多少 warps 可以同时驻留。高 occupancy 能隐藏 memory latency，因为一个 warp 等 HBM 时，SM 可以切到另一个 ready warp。但 occupancy 太高不一定最好：如果每个 thread 使用更多 registers 来做更多 work，低 occupancy 也可能换来更高 arithmetic intensity。

occupancy 计算直觉

如果每个 thread 用 160 个 registers，一个 block 有 128 threads，那么一个 block 需要 $160 \times 128 = 20480$ registers。若一个 SM 有 65536 registers，最多只能同时放下 3 个这样的 blocks。register pressure、block size 和 max warps 共同决定 occupancy。

11.2 Bank conflicts 与 swizzling

Shared memory 被分成多个 banks。若同一 cycle 内多个 threads 访问同一个 bank 的不同地址，访问会被串行化，形成 bank conflict。矩阵乘法中，读取 A 的行和 B 的列很容易造成不同访问模式，因此高性能 kernel 常通过 swizzling 改变 shared memory layout，减少冲突。

bank conflict 很隐蔽

代码语义完全正确，但性能可能因为 bank conflict 急剧下降。Profiler 或更底层的 Nsight 指标才能确认这类问题。只看 Python 层时间很难定位。

11.3 Memory coalescing 与 HBM transaction

HBM 访问通常以 cache line / transaction 为单位。若一个 warp 的 threads 访问连续地址，硬件可以合并成少量 transaction；若访问跨越很多不连续位置，就会浪费带宽。Coalescing 是 global memory 访问优化的第一原则。

coalescing 的一句话判断

同一个 warp 的相邻 threads 最好访问相邻地址。若 thread 0、1、2、... 访问的是矩阵同一行的连续元素，通常更 coalesced；若访问同一列且 row stride 很大，往往更差。

12 Triton kernel 参数表

参数/概念	出现位置	含义
-------	------	----

BLOCK_SIZE	GeLU、softmax、row sum	一个 program 处理多少元素或列。太小复用差，太大 register/shared memory 压力高。
tl.program_id	所有 Triton kernels	当前 program/block 的 id，用来决定处理哪一块数据。
offsets	GeLU/softmax	当前 block 对应的全局元素下标。
mask	尾部处理	防止读写越界，是 block size 不整除数据长度时的必要保护。
stride	softmax/matmul	描述逻辑二维 tensor 在一维内存中的跳步方式。
BLOCK_M/N/K	matmul	C tile 的行/列和 K 维 chunk 大小，决定 tile shape 和复用。
tl.dot	matmul	调用 Triton 的 block-level dot，通常映射到底层矩阵硬件。

为什么这些参数不是随便调

Triton kernel 的参数直接映射到硬件资源：block size 影响并行粒度，mask 影响边界处理，stride 决定内存访问连续性，tile shape 影响 shared memory/register pressure 和 tensor core 使用。调参不是玄学，而是在硬件约束下找平衡。

13 扩展讲解：从源码到性能判断

13.1 为什么 GeLU 是第一个好例子

GeLU 是 elementwise operation，数学上不复杂，却非常适合说明 kernel fusion。因为 naive GeLU 由乘法、加法、立方、tanh、再乘法组成，如果框架把它拆成多个 kernels，每个中间 tensor 都要写回 HBM 再读出。

版本	执行方式	资源结果
naive PyTorch	多个 pointwise kernels	launch 多，HBM 往返多。
builtin GeLU	手写/库内 fused kernel	读一次、算完、写一次。
compiled GeLU	torch.compile 生成 Triton kernel	保持 Python 表达，获得 fusion。

elementwise kernel 的核心瓶颈

Elementwise 操作的 FLOPs 很少，bytes moved 却和 tensor 大小成正比。单独执行时通常 memory-bound。Fusion 的目标不是减少数学 FLOPs，而是减少中间结果在 HBM 中来回搬运。

13.2 Profiler 表应该怎么看

当 profiler 显示很多小 kernel 时，通常有三种解释：

1. Python/PyTorch 代码被拆成多个 primitive operations。
2. 每个 primitive operation 都 launch 一个独立 CUDA kernel。

3. 中间结果 materialize 到 HBM, 形成额外 memory traffic。

若 profiler 显示一个长名字的 CUTLASS/Triton kernel, 则要读名字里的线索: 架构代号、dtype、tile shape、layout alignment。这些信息帮助判断是不是 tensor core kernel、tile 是否过小、是否有 alignment 问题。

profile 后怎么行动

如果瓶颈是许多小 pointwise kernels, 优先考虑 fusion; 如果瓶颈是 GEMM 但 MFU 低, 检查 shape、dtype、layout、tile; 如果瓶颈是 memory copy 或 communication, 优化 kernel 本身可能无效。

13.3 Triton masks 的必要性

很多 tensor 长度不能被 block size 整除。例如 $N = 10,000$, $BLOCK_SIZE = 1024$, 最后一个 block 会越界。Triton 用 mask 防止读写越界:

Listing 7: mask 保护尾部 block

```
1 offsets = start + tl.arange(0, BLOCK_SIZE)
2 mask = offsets < num_elements
3 x = tl.load(x_ptr + offsets, mask=mask, other=0.0)
4 tl.store(y_ptr + offsets, y, mask=mask)
```

mask 是 correctness 条件, 不只是性能细节

没有 mask, 最后一个 block 可能读到非法地址或写坏输出。很多 Triton kernel 的第一类 bug 就来自 tail block 没处理好。

13.4 Softmax 的读写账本

Naive row softmax 至少经历这些阶段: row max、subtract max、exp、row sum、divide。若每步都是独立 kernel, 读写次数近似为多次 MN 。Fused row-wise softmax 则把一整行放进 block 中, 局部完成 reduction 和 normalize。

阶段	naive 实现	fused Triton 实现
max	读整行, 写 max	block 内 reduction。
subtract/exp	再读整行, 写中间结果	registers/shared memory 中完成。
sum	再读 numerator, 写 denominator	block 内 reduction。
divide	再读 numerator/denominator, 写输出	只写最终输出。

13.5 Matmul tiling 的指针结构

Triton matmul kernel 不是直接写三重 for loop, 而是构造指针矩阵:

- indices_m 选择 C tile 的行。
- indices_n 选择 C tile 的列。

- `indices_k` 选择当前 K tile。
- `a_ptrs` 和 `b_ptrs` 指向当前要加载的 A/B tile。

每轮 K tile 做一次 `tl.dot(a,b)`，累加到 `acc`。循环结束后可以在 `acc` 上做 ReLU，再写回 C。

Tiled matmul 的统一解释

Tiling 把 HBM 里的大矩阵切成可放入片上存储的小块，让一个 tile 内的数据服务多个输出元素。Fusion 则把 matmul 后的 ReLU 放在写回前完成。两者都在减少 HBM 访问。

13.6 本章小结

源码中的每个 kernel example 都在回答同一个问题：如何把高层 tensor expression 改写成更少 HBM traffic、更高片上复用、更适合 SM/warp/block 执行的程序。Triton 的价值在于让这种改写比 CUDA 更接近数学表达，但仍保留 block-level 控制。

14 Assignment 2 视角：从 correctness 到 performance

Lecture 6 对应的 systems assignment 不只是“写出能跑的 kernel”，而是要求你 benchmark 和 profile 实现。一个 kernel 的评价至少包含三层：

层级	问题	证据
Correctness	输出是否和 PyTorch reference 接近？	<code>torch.allclose</code> 、单元测试、极端 shape。
Performance	是否比 baseline 快？随 shape 如何 scaling？	CUDA event benchmark, 多次 trial, warmup。
Attribution	时间花在哪些 kernels 和 memory 操作上？	PyTorch profiler、Nsight、kernel 名、occupancy/throughput 指标。

不要跳过 correctness

高性能错误 kernel 没有意义。Triton 的 mask、stride、边界条件、dtype accumulation 都可能造成 subtle bug。Assignment 的正确姿势是先让 reference 对齐，再 benchmark，再 profile，再优化。

性能比较要公平

比较 naive、builtin、compiled、Triton 版本时，必须使用相同输入 shape、相同 dtype、相同 device，并包含 warmup。否则测到的可能是编译时间、CPU launch overhead、allocator 行为或缓存状态，而不是 kernel 本身。

14.1 从本讲走向下一讲

本讲所有优化仍在单 GPU 内部。下一讲进入多 GPU 后，同样的原则会扩大：HBM 往返变成 GPU 间通信，thread blocks 变成 ranks 和 collectives，kernel fusion 的“少搬数据”变成 all-reduce、reduce-scatter、all-gather 的通信账本。

单 GPU kernel 优化和多 GPU parallelism 都是在问：数据在哪里、谁需要它、什么时候移动、能不能少移动或多复用。Lecture 6 学的是这个问题在一个 GPU 内部的版本。

15 总结与延伸

本讲把 GPU kernel 工作流压缩成一条路径：理解硬件层级，写出正确 kernel，benchmark 测端到端时间，profile 找真实 kernel 和瓶颈，再用 Triton 调整 block、mask、tiling 和 fusion。

最终 takeaway

性能优化不是玄学。每一次优化都应该能回答：减少了哪次 HBM 读写？增加了多少复用？提高了 occupancy 还是降低了 bank conflict？benchmark 和 profiler 是否支持这个判断？

15.1 拓展阅读

- Triton fused softmax tutorial.
- NVIDIA CUDA programming guide: warps, shared memory, coalescing.
- CUTLASS documentation for GEMM tiling and tensor core kernels.
- PyTorch profiler and Nsight Systems/Compute guides.